Computer Science Does a body good

CARTHAGE COLLEGE

Discourses and Dialogs on Debugging

Perry Kivolowitz

Professor of Computer Science

Carthage College

Summary

Megan and Fredo wrote buggy code.

Megan paid attention to this talk, found her problems quickly and went out dancing.

Fredo is still slamming Red Bulls.

Fredo can’t feel his lips.

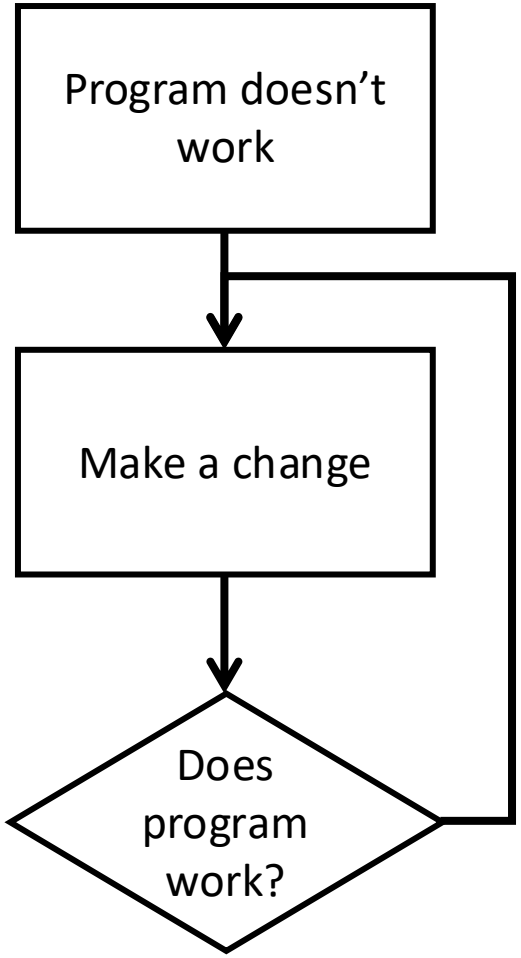
Don’t be like Fredo.

The thesis of this talk is:

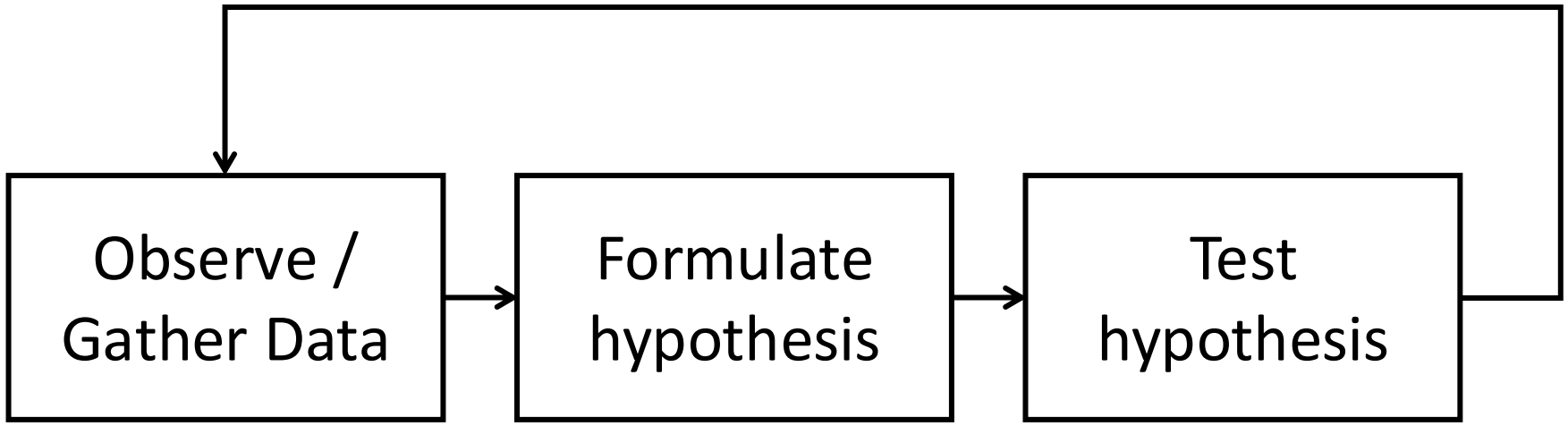
debugging is based in *science*

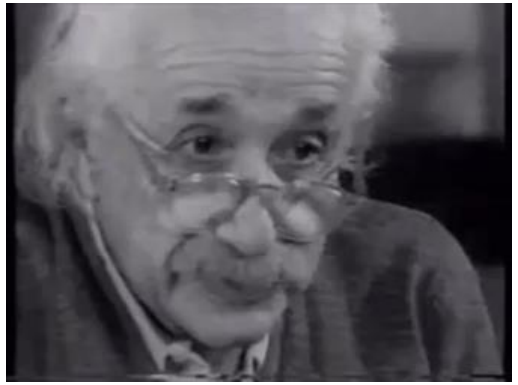
(science can be taught)

Most Common Debugging Algorithm



The Scientific Method





Albert sez:

Debugging should be methodical

Kivolowitz Corollary

"A fix is **not a fix** until you completely **understand why** it's a fix"

Kivolowitz Maxim #1

If you make a change with no beneficial result, back it out*

Else, you are chasing a moving target

*unless you are certain you're fixing a different bug

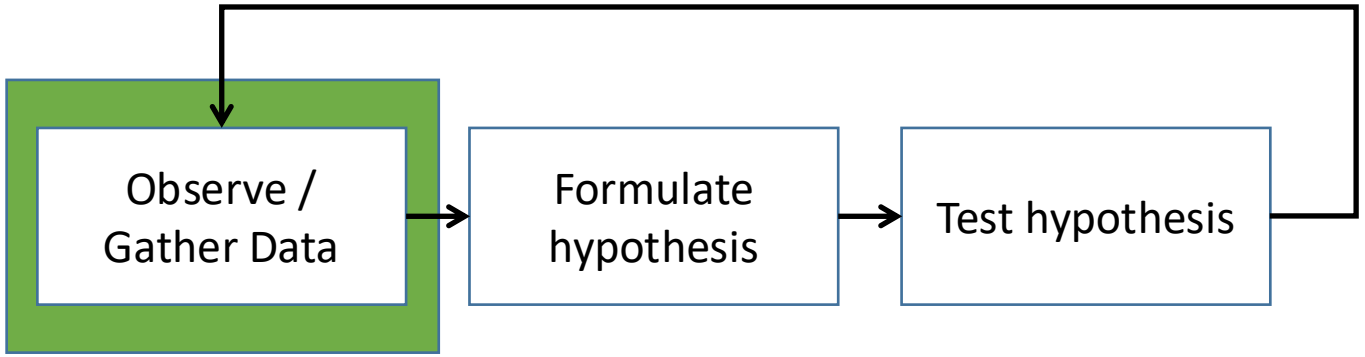
Maxwell (Mickey) Cohen's Law

"Where there is one, there are many"

Kivolowitz's Maxim #2

Bugs want to be found

if you make the effort to listen



Conan Doyle's Law

"When you have eliminated the impossible, whatever remains, however improbable, must be the truth."

The Sign of Four, 1890

Kivolowitz Maxim #3

“Write a little.
Test a little.”

Kivolowitz Maxim #4

“Always play defense”

Extra work is finite.
Debugging time…
who knows

Part 2 - Dialogs *(all actually happened)*

http://images6.fanpop.com/image/photos/33100000/Kung-Fu-Panda-3-random-33170400-1920-810.jpg

Kivolowitz Corollary Example

Student: My code crashed. I added a printout. Now it doesn't crash. It's fixed right?

No understanding of the "fix"

Kivolowitz's Maxim #2 Example

Student: So, I have this error.

Master: What does the error say?

Student: I didn't read it. I just clicked "OK"

"Bugs want to be found"

They often announce themselves clearly

if you make the effort to listen

Kivolowitz's Maxim #2 Example

Student: My code crashes.

Master: You get compiler warnings.

Student: They're just warnings.

"Bugs want to be found"

They often announce themselves clearly

if you make the effort to listen

Kivolowitz's Maxim #2 Example

Student: It crashes after running for a while.

Master: Do you have a memory leak?

Student: No, I already went.

"Bugs want to be found"

Use leak detection tools.

Leave code running for ____ if warranted

Kivolowitz's Maxim #2 Example

Student: After a while my program stops responding.

Master: Do you have any infinite loops?

Student: No, I use "whiles."

"Bugs want to be found"

Not responding? Check CPU utilization.

In a loop first code to write is how it exits.

Kivolowitz's Maxim #2 Example

Student: My code works great except on this input.

Master: Try single-stepping with just that.

Student: Single… what?

"Bugs want to be found"

Your environment should be your friend.

Get to know it – USE IT!

Conan Doyle's Law

Student: I looked everywhere for my bug.

Master: Did you look here?

Student: No. It can't possibly be there.

"When you have eliminated the impossible, whatever remains, however improbable, must be the truth."

Defensive programming *and* Doyle's Law

Student: I looked everywhere for my bug.

Master: Did you look here?

Student: No. It can't possibly be there.

If you "know" a condition to be true, `assert()` it!

Stops a cascade in its tracks

Checks the impossible and identifies the improbable

Cohen's Law, Defensive programming *and* Doyle's Law

```
#include <assert.h>

assert(condition I know to be true);
```

assert enforces "contracts"

Kivolowitz Maxim #3 Example

Student: My program worked yesterday. I wrote 5000 lines of code last night. It doesn't work anymore.

Why?

Write a little.

Test a little.

Maxim #4 - Defensive programming example

Student: I commented like you told me.

```
// Increment i
```

Comment your thinking.

Not your code.

Maxim #4 - Defensive programming example

Student: I commented like you told me.

```
// Increment i
```

When commenting, think of the next person to read your code. It might be you.

Maxim #4 - Defensive programming example

Student: My code doesn't work.

Master: And there is no way it ever would. Did you test it *before* you wrote it?

Student: Wait… What?

Write the comments **before** your code

Maxim #4 - Defensive programming example

Student: My code doesn't work.

Master: What does `ineedsleep` do?

Student: I don't remember.

I was tired.

`descriptive_variable_names_really_help`

Keystrokes are finite. Debugging time isn't.

Maxim #4 - Defensive programming example

Student: ____; ____; ____; ____;

Master: What's that in the middle?

Student: Where?

Newlines are free. Resist temptation for multiple statements on one line.

Maxim #4 - Defensive programming example

Student: `__ += __ ? (__ = --__ , __) : __ = __;`

Master: What’s that in the middle?

Student: Where?

Use side effects with caution

Optimizers laugh at your lame attempts to be cool.

Your coworkers just get annoyed.

Defensive programming *and* Doyle's Law

Student: The bug can't be here.
I pasted this code.

Master: Really?

That's different from this.

Student: Oh yeeaahhhh. I fixed a bug.

Code abstraction is actually a defensive technique

Code in one place, test in one place, fix in one place

Defensive programming *and* Doyle's Law

Student: foo has to be one of these values.

Master: And if it isn't?

Student: That's not possible.

Always code the "default" case

Always code the "impossible else"

(tell Ben Liblit's story)

Maxim #4 - Defensive programming example

Student: My code doesn't work.

Master: You're not checking that return value

Student: That has a return value?

Return values are meant to be checked

Importance of checking return values

- Need go no further than `su*`
- Runs setuid to root

```
-rwsr-xr-x 1 root root 36936 Jan 26  2016 su
```

- Suppose it was coded like this:

```
setuid(uid);
execlp(argv[2], argv[2], NULL);
```

*Thank you Bart Miller

Maxim #4 - Defensive programming

“Dying is the most embarrassing thing that can ever happen to you, because someone's got to take care of all your details.”

Andy Warhol - "The Life and Death of Andy Warhol" by Victor Bockris

Lesson for programmers:

Take care of all your OWN details and die gracefully.

One last dialog

Student: I crash every time I get here.

Master: That’s GREAT!

Student:

Learn your craft - Take pride in your work

Debugging is part of the process

Grow your craft

Take pride in your work

Code like it means the difference between life and death - it may